Who Sheds? A Kubernetes Power-Budget Operator for Multi-Service LLM Inference

Karthik Mattu
Rochester Institute of Technology
km5503@rit.edu

Abstract

New AI datacenters wait years in grid interconnection queues; sites that can shed load on request get connected sooner. Training jobs can pause, but live LLM inference cannot: every watt shed is a user waiting. Recent work shows a single GPU server can follow a power target while serving traffic, but no open system decides, for a whole cluster, *which* nodes, pools and services should shed, in what order, and whether the total met the grid's requirement. We propose a Kubernetes operator that turns one grid power target into per-node budgets by merit order (cheapest SLO cost per watt first), enforces them only on capacity whose tenants consented to slow down, and reports time-to-depth and hold accuracy. A preliminary study on a 12-node model with published lever curves shows that a uniform cut costs 2–10$\times$ the optimum once nodes differ, while greedy merit order stays within 18% of a mixed-integer optimum. The system will be evaluated on GKE with one real T4 GPU node and four emulated nodes whose scaling, admission and drain actions are real Kubernetes operations.

1 Introduction

Large new loads are arriving faster than the grid can connect them. AI datacenters of 100 MW and more face multi-year interconnection queues, and studies of US systems find that loads willing to curtail for a small fraction of hours can be connected far sooner and in much larger aggregate than rigid loads [1, 15]. Grid operators are starting to condition new capacity on exactly this kind of demand response (DR) [10], and recent field trials have shown GPU clusters curtailing 25% for three hours on a utility signal [2, 20].

Those trials shed mostly *training* load, which tolerates pausing. Live *inference* is harder: a survey of the area calls synchronous inference "effectively non-dispatchable" [14]. Every watt taken from a serving GPU shows up as latency for a user who is waiting. And a production cluster is not one workload. It runs many services, some with strict latency service-level objectives (SLOs) and some that have agreed to be deferred, on nodes that differ in load and in what each phase of inference costs in power. Cutting every node by the same fraction is therefore the wrong answer; the question is who sheds, and by how much.

Question. Given a cluster-wide power budget and services with different latency tiers, which nodes and pools should shed, in what order, and does the control loop reach and hold the target on Kubernetes without breaking premium SLOs?

Contributions. This project will deliver (1) **an artifact**: an open, reusable Kubernetes operator that turns a grid power target into per-node budgets for LLM inference, the layer missing above today's node-level controllers; and (2) **a finding**: whether that loop reaches the target within a 10-minute deployment window and holds it under real Kubernetes actuation and telemetry delays, and what it costs premium users.

2 Background and Related Work

What the grid asks for. A DR obligation is a tuple: respond within a deadline, reach a depth (MW), hold it for a duration, and be scored over metering intervals. The 5-minute number often quoted is the *settlement* interval of real-time markets [4], i.e. how performance is metered, not how fast a load must react. Response deadlines vary by product (Table 1). Sub-second frequency response is out of reach for any cluster-level software loop, since metric scraping alone takes 15–60 s; that regime belongs to hardware power caps on each node. Emergency and contingency products with 10–30 minute deadlines are the natural target for a cluster controller, and we adopt the 10-minute window.

Node-level power control of inference. LLM inference has two phases: *prefill* processes the prompt and is compute-bound; *decode* generates tokens one at a time and is memory-bound [17, 22]. Consequently, lowering a GPU's clock slows

Product (ERCOT [3])	Respond within	Cluster loop?
Fast frequency response	$<$1 s, automatic	no
Contingency reserve (ECRS)	$\approx$10 min	yes, tight
Emergency Response Service	10 or 30 min	**target**

Table 1: Response deadlines of grid products a large load can join. A cluster-level loop can meet the minute-scale ones.

prefill roughly in proportion but leaves decode almost unaffected down to about 0.57$\times$ nominal frequency [10], and power caps act mainly on the prompt phase [16]. PowerSlider [10] exploits this asymmetry to follow a time-varying cap with per-phase DVFS and admission control; PALS [5] co-tunes caps and batch size in vLLM [8] to track a DR signal; Yan et al. [21] remove a decode instance from live H200 serving, saving $\approx$620 W at the cost of median TTFT rising from 0.7 s to 8.5 s; RAPID [6] reallocates power between prefill and decode pools under a fixed cap; and TAPAS [19] reroutes inference during a power emergency. All assume root access to the GPU and decide for one server or one pool.

Cluster- and fleet-level control. Sreekumar et al. [18] meet time-varying caps across 15 k simulated servers by separating slow server transitions from fast throttling. GPU-to-Grid [11] couples a fleet's batch size to grid voltage in simulation. ETCInfer [12] runs nearly our stack (Kubernetes, KServe, vLLM, DCGM) but minimises energy rather than tracking a target, and Azure's carbon-aware KEDA operator [13] caps replicas from a carbon forecast without closing the loop on measured power. Field demonstrations of cluster-level curtailment exist [2,20], but their allocation method is not published. **Gap:** no open system allocates one power budget across heterogeneous nodes, pools and service tiers on Kubernetes for inference, closes the loop on measured power, and reports whether the grid requirement was met.

3 Technical Approach

Figure 1 shows the system. It follows the Kubernetes operator pattern: desired state lives in a custom resource, and a controller reconciles the cluster toward it.

Declare. A replay of ERCOT/CAISO price and emergency signals publishes events to Pub/Sub; a small consumer turns each event into a `PowerBudget` custom resource holding the target (fraction of cluster nominal power), the window, the premium p99 latency to protect, and a tolerance.

Decide. The operator, written with kopf, reconciles every few seconds. It reads measured cluster power from Prometheus and builds, for each node, a menu of options: (lever, watts shed, SLO cost), including "do nothing". The levers are admission control by tier, draining a pool to idle, changing a pool's replica count, and, on nodes where the operator has root, a power cap or clock lock. Each option's watts and latency cost come from *lever curves*: published measurements [10, 16, 21] until our companion measurement project supplies A100 curves in the same JSON schema. The operator ranks every node$\times$pool$\times$lever upgrade by marginal SLO cost per watt reclaimed and takes the cheapest until the gap to target closes, writing a *per-node budget* into the resource's status. This is economic dispatch with SLO-cost curves in place of heat-rate curves; Section 4 shows greedy merit order is near-optimal, so the allocator is deliberately simple. Lead times enter the ranking: a replica removal lands in about 90 s, so fast levers (admission, caps) cover the first minutes and slow ones (replicas) sustain depth. Hysteresis and ramp-limited recovery prevent oscillation and rebound.

Act, within authority. A node agent spends its budget with the levers its node has. Grid studies already price firm and flexible load tiers differently [7]; we carry that distinction inside the cluster. An operator may slow only capacity whose tenants consented to deferral, and never degrades request semantics (e.g. by quantization or reasoning effort). Budgets are therefore *binding* on flex-tier capacity and *advisory* on premium services; the shortfall from advisory nodes is reported, never forced.

Sense. The DCGM exporter publishes GPU power; vLLM publishes time-to-first-token (TTFT), time-per-output-token (TPOT) and queue depth; GKE Managed Prometheus scrapes both. A Grafana burn-rate alert on premium p99 latency calls the operator, which backs off.

Testbed. Terraform provisions a GKE cluster and tears it down between sessions. *One real node*, a T4 Spot GPU running vLLM, exercises the full chain with a real power meter. *Four emulated nodes* are Spot CPU nodes, each running a mock serving pod that answers the load generator's real requests with latency computed from the lever curves for its current state, and a sidecar that publishes a DCGM-shaped power metric. On these nodes the replica, admission and drain levers are real Kubernetes actions: pods terminate and start, the proxy rejects requests, and queues grow on the surviving pods. Only the watt reading and the cap/clock lever are modelled. The emulated cluster therefore tests the *loop* (reconcile timing, scrape lag, pod lead times, alert delay, overshoot, rebound) under real orchestration, not the GPU physics. Cost is about $0.35/h all-Spot.

Metrics and baselines. Per DR event and policy we report *time to depth* (requirement $\leq$10 min), *hold accuracy* (each 5-minute interval average within tolerance of the target), premium p99 held or not, rebound after release, and advisory

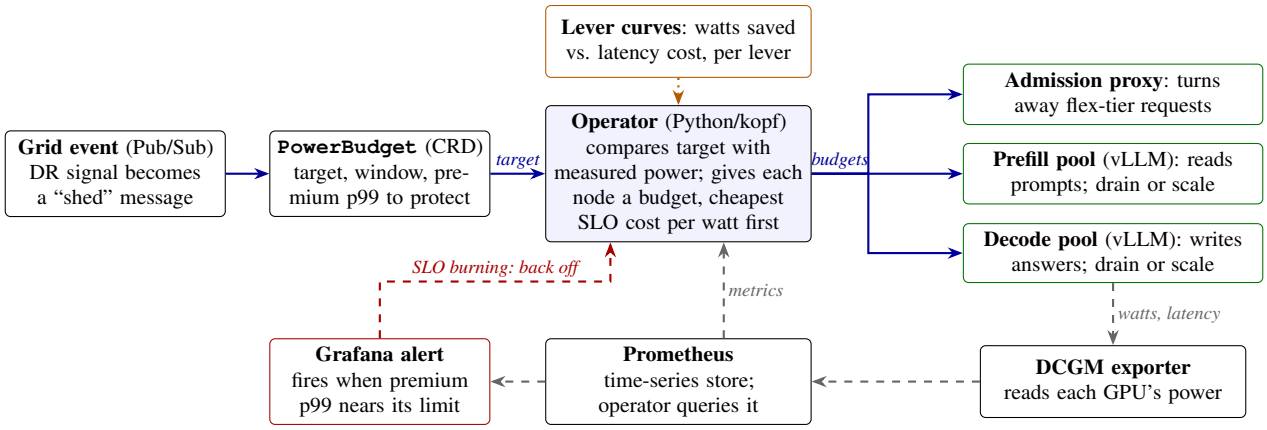


Figure 1: The control loop, repeated every few seconds while a budget is active. Solid: control actions; dashed: telemetry; red: SLO guard; dotted: lever-curve input.

Nodes differ in…	Opt.	Unif./opt.	Greedy/opt.
nothing (identical)	0.94	1.03	1.06
service tier	0.20	2.89	1.12
load	0.74	1.31	1.00
phase, decode flat	0.58	1.68	1.00
phase, decode half-sens.	1.31	1.11	1.00
all three	0.13	4.47	1.00

Table 2: One source of heterogeneity at a time, cut B=0.30. Costs are weighted SLO cost; ratios are relative to the MIP optimum.

shortfall. Baselines are a uniform cut and a monolithic cluster (prefill and decode on the same GPUs), both driven by a tiered load generator replaying an identical trace.

4 Preliminary Results

Before building the allocator we tested whether allocation matters at all. We modelled a 12-node cluster in which each node picks one option from its lever menu (a multiple-choice knapsack), with lever curves taken from the literature: PowerSlider's A100 power–frequency relation, decode latency flat down to a clock knee, Yan et al.'s instance-removal penalty, and early estimates from the companion project. The objective is tier-weighted relative latency increase (premium 1.0, flex 0.2). A heterogeneity knob h moves nodes from identical (h=0: all monolithic, premium, 70% load) to mixed (h=1: half prefill and half decode, half hosting a flex service, load from 35% to 95%). We compare a uniform cut, greedy merit order, the LP relaxation, and the exact optimum (MIP, solved with SciPy's HiGHS interface).

Figure 2 and Table 2 give three results. **(1) Allocation matters.** Once nodes differ, a uniform cut costs 2–10× the optimum. **(2) Greedy suffices.** Merit order stays within 18% of the MIP for every cut below 35%, and the LP bound sits within 5% of the MIP, so discreteness is not where the problem is hard. **(3) Tier consent is the largest driver** (2.9× on its own), followed by load. Prefill/decode separation lowers the optimal cost by ≈40% if decode is as clock-insensitive as PowerSlider reports, but *raises* it by ≈40% if decode is only half as insensitive; whether disaggregation helps DR therefore hinges on one measurable slope, in line with findings that split serving saves no energy on small PCIe GPUs [9].

These are model results. Removal cost uses pool utilization at the start of an interval, which keeps the problem separable; on a live cluster removing one replica loads the others, and measuring that coupling under real delays is the point of the testbed. Only the ratios are claimed, not absolute costs.

5 Milestones

1. **Cluster and telemetry.** Terraform for GKE, Spot T4 and CPU pools, Pub/Sub and IAM; DCGM → Managed Prometheus → Grafana showing live T4 watts. *Done when* apply/destroy round-trips cleanly.
2. **Serving and load.** vLLM on the T4; tiered load generator with trace replay. *Done when* per-tier p99 appears in Grafana.
3. **CRD and operator.** `PowerBudget` resource with per-node status; kopf operator; node agent; admission proxy. *Done when* a hand-applied budget lowers T4 power and recovers.
4. **Emulated nodes.** Mock serving pod and power sidecar on four CPU nodes. *Done when* the operator cannot distinguish them from the T4.
5. **Allocator and reporting.** Merit order with uniform and monolithic baselines; event report (time to depth, interval

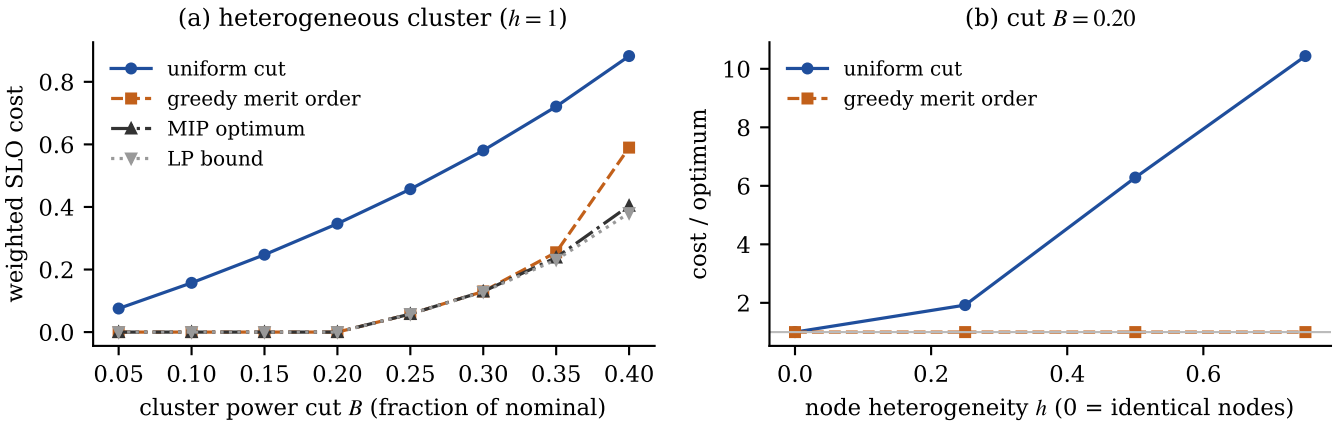


Figure 2: Stage 0 allocation study (12-node model, published lever curves). (a) On a heterogeneous cluster, greedy tracks the optimum and sheds up to 20% at zero SLO cost; a uniform cut degrades latency from the first watt. (b) With identical nodes uniform is optimal; as nodes differ it reaches 6–10× the optimal cost, while greedy stays at 1.0.

accuracy, shortfall). *Done when* the report reconciles with per-node logs.

6. **SLO guard and experiments.** Alert-driven back-off; full matrix of policy × depth × trace; final report.

Risks. If T4 quota is not granted, the emulated cluster carries every result except the real-power wiring. If measured A100 curves do not arrive, the project is complete on published curves, labelled as such. The LP allocator is a stretch goal; Section 4 suggests greedy leaves little on the table.

References

[1] Yize Chen and Xiaogui Zheng. To defer or to shift? the role of AI data center flexibility on grid interconnection. arXiv:2604.05376, 2026.

[2] Philip Colangelo, Ayse K. Coskun, Jack Megrue, Ciaran Roberts, Shayan Sengupta, Varun Sivaram, Ethan Tiao, Aroon Vijaykar, Chris Williams, Daniel C. Wilson, Zack MacFarland, Daniel Dreiling, Nathan Morey, Anuja Ratnayake, and Baskar Vairamohan. Turning AI data centers into grid-interactive assets: Results from a field demonstration in Phoenix, Arizona. arXiv:2507.00909, 2025.

[3] Electric Reliability Council of Texas. ERCOT grid insights: Demand response. https://www.ercot.com/files/docs/2025/12/22/ERCOT-Grid-Insights-Demand-Response.pdf, December 2025.

[4] Federal Energy Regulatory Commission. Order no. 825: Settlement intervals and shortage pricing in markets operated by regional transmission organizations and independent system operators. 155 FERC ¶ 61,276, 2016.

[5] Can Hankendi, Rana Shahout, Minlan Yu, and Ayse K. Coskun. PALS: Power-aware LLM serving for mixture-of-experts models. arXiv:2605.21427, 2026.

[6] Yiwei Jiang, Sangeeta Chowdhary, Nathaniel Morris, Rutwik Jain, Srilatha Manne, and Samuel R. Bayliss. Power aware dynamic reallocation for inference. arXiv:2601.12241, 2026.

[7] Saroj Khanal, Geon Roh, Boyu Yao, Abraham Silverman, Dennice Gayme, Charalambos Konstantinou, Jip Kim, and Yury Dvorkin. Shift or curtail? how much datacenter flexibility is worth depends on the host power grid. arXiv:2608.19622, 2026.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[9] Jiaxi Li, Yue Zhu, Eun Kyung Lee, and Klara Nahrstedt. Revisiting disaggregated large language model serving for performance and energy implications. arXiv:2601.08833, 2025.

[10] Yueying Li, Jiayang Chen, Yuanfan Chen, Leo Han, Haoran Qiu, Esha Choukse, Rodrigo Fonseca, and Udit Gupta. PowerSlider: Exploiting phase asymmetry for LLM serving under demand response. arXiv:2608.21719, 2026.

[11] Zhirui Liang, Jae-Won Chung, Mosharaf Chowdhury, Jiasi Chen, and Vladimir Dvorkin. GPU-to-Grid: Voltage regulation via GPU utilization control. arXiv:2602.05116, 2026.

[12] Rui Lu, Rui Ge, Huanghuang Liang, Xiaobo Zhou, and Dan Wang. ETCInfer: An energy-efficient thermal-aware cooling-joint scheduler for LLM inference in AI datacenters. arXiv:2609.15230, 2026.

[13] Microsoft Azure. Carbon aware KEDA operator. https://github.com/Azure/carbon-aware-keda-operator, 2023.

[14] Mariam Mughees, Yuzhuo Li, Yize Chen, and Yunwei Ryan Li. AI data centers meet electrical grids: A review of power challenges and coordinated solutions. arXiv:2610.09060, 2026.

[15] Tyler H. Norris, Tim Profeta, Dalia Patiño-Echeverri, and Adam Cowie-Haskell. Rethinking load growth: Assessing the potential for integration of large flexible loads in US power systems. Technical report, Nicholas Institute for Energy, Environment & Sustainability, Duke University, February 2025.

[16] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. Characterizing power management opportunities for LLMs in the cloud. In *Proc. 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 207–222, 2024.

[17] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proc. 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024.

[18] Akshay Sreekumar, Nicolas Christianson, Fiodar Kazhamiaka, and Ram Rajagopal. Multi-scale datacenter power modulation. arXiv:2609.17809, 2026.

[19] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. TAPAS: Thermal- and power-aware scheduling for LLM inference in cloud platforms. arXiv:2501.02600 (ASPLOS '25), 2025.

[20] Chris Williams, Philip Colangelo, Ayse K. Coskun, et al. Power-flexible AI data centers: A new paradigm for grid-responsive compute. arXiv:2606.25098, 2026.

[21] Mingyuan Yan, Haiyu Wang, Linxuan Biao, H. Jonathan Chao, Sai Qian Zhang, and Wenqi Cui. Analytical power-aware provisioning for prefill–decode disaggregated AI inference. arXiv:2609.24639, 2026.

[22] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.